Passing Tests, Missing Budgets

TempO / RotCodeBench: Measuring Budget Blind Spots in Coding Agents

Yuanbo Pang | UC Berkeley EECS | Berkeley RDI/BAIR **Ongoing work; manuscript under review. Please do not distribute.**

Research question. A coding agent can complete a feature request and pass functional tests while silently violating an existing performance contract. Can agents recognize resource constraints that are *implicit in the repository*, and can execution traces and verifiable feedback help them act on those constraints?

Benchmark and Evaluation

RotCodeBench is a challenge set of real, merged feature changes. Each task gives an agent only the pre-change repository and a **performance-silent functional request**. Hidden, rule-based black-box oracles first check functionality and then measure a task-specific cost contract, such as operation or render counts, allocations or bytes, network requests or writes, and, in two strongly separated cases, timing. The engineers' upstream patch establishes an attainable budget; a deliberately naive implementation validates that the oracle detects a regression.

17	**7**	**442**	**8 / 26**
real-repository tasks	programming languages	configuration–task trials	models / configurations

Every output receives one of three interpretable verdicts: **efficient**, **correct but over budget** ("performance rot"), or **functionally incorrect**. Most cost oracles use discrete, non-timing signals and **none uses an LLM judge**, making the resulting feedback auditable and suitable as a verifiable training signal.

Core Findings

- **Functional success does not imply budget awareness.** Across the scored grid, **148 of 349 functionally correct solutions (42%)** exceed the calibrated performance budget; per-model rot rates range from **23% to 51%**. Functional pass rates of 0.65–0.94 fall to 0.24–0.71 when the cost contract is also required.
- **The primary failure is recognition, not retrieval.** In a trajectory audit of all 148 cost-gap runs, agents reach the relevant mechanism in **147/148** cases, but **145/148** contain no explicit connection between that evidence and cost. Agents often see the right code without recognizing the constraint it encodes.
- **A minimal constraint cue substantially closes the gap.** Adding one sentence that names the task-specific performance invariant, without naming a file, API, metric, or implementation, reduces rot from **46% to 8%** and raises efficient pass from **44% to 67%** across 102 model–task pairs and four providers.

From TempO to Budget-Aware Agents

TempO studies the complementary, execution-side question by mining Terminal-Bench trajectories for wasted tool-call time: repeated slow calls, over-waiting, under-wait churn, repeated errors, and redundant environment setup. Together, the two views distinguish **the cost of running the agent** from **the downstream cost of the artifact it produces**.

Connection to BAGEN and agentic RL. BAGEN formalizes budget awareness as *progressive interval estimation*: from a partial rollout, an agent predicts bounds on its remaining resource budget and alerts when completion is unlikely. A natural next step is to apply this idea to coding-agent trajectories and jointly estimate **execution cost, completion risk, and the eventual performance cost of the generated patch**. RotCodeBench's rule-based oracles can supply **verifiable rewards for RL/RLVR**, while TempO-style trajectory signals can support earlier diagnosis and intervention.

Relevant background. At Berkeley RDI/BAIR, I am a co-first author and core contributor to *Agents' Last Exam* (ALE, arXiv:2606.05405), where I help build scalable agent evaluation and programmatic evaluators that produce verifiable rewards for RLVR. **BAGEN reference:** *Are LLM Agents Budget-Aware?* (arXiv:2606.00198).